Zomato App Play Store Reviews

Data Processing Pipeline :

- **Analytical Platform**
- **Data Ingestion**
- **Data Cleaning**
- **Univariate Analysis**
- **Bivariate Analysis**
- **Multivariate Analysis**
- **Feature Engineering**
- **Machine Learning**
- **Implement and Evaluate Regression Models**
- **Visualize Regression Models Performance**
- **Select and Visualize Best Regression Model**

Analytical Platform

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported: matplotlib.pyplot as plt, seaborn as sns")
```

```
Libraries imported: matplotlib.pyplot as plt, seaborn as sns
```

Data Ingestion

In [2]:
```
df = pd.read_csv('D:/PYTHON 2/DATA/DECEMBER/Zomato App Play Store Reviews/zomat
display(df.head())
```

	review_id	rating	review_text	review_date	helpful
0	90749778-cd88-4c19-8b12-1fce7e7d82f8	4	kindly requesting to return change . we are fo...	2025-11-27 08:15:26	0
1	aa848bb6-d242-4a7e-831e-4f21e2e60c6e	1	Hiked prices, packing and platform charges	2025-11-27 08:08:31	0
2	4f888388-9f28-44a4-8601-491a87035e53	5	good discount	2025-11-27 04:20:28	0
3	490a16b3-aacf-4204-bdcb-ffdbf04add72	1	Zomato in its initial days was too good, but c...	2025-11-27 03:34:38	0
4	0090a503-13b8-4741-a7c0-42e811244563	5	good application	2025-11-27 02:50:58	0

Data Cleaning

In [3]:
```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
review_id      0
rating         0
review_text    0
review_date    0
helpful        0
dtype: int64
```

In [4]:
```
print("Number of duplicate rows before removal:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Number of duplicate rows after removal:", df.duplicated().sum())
```

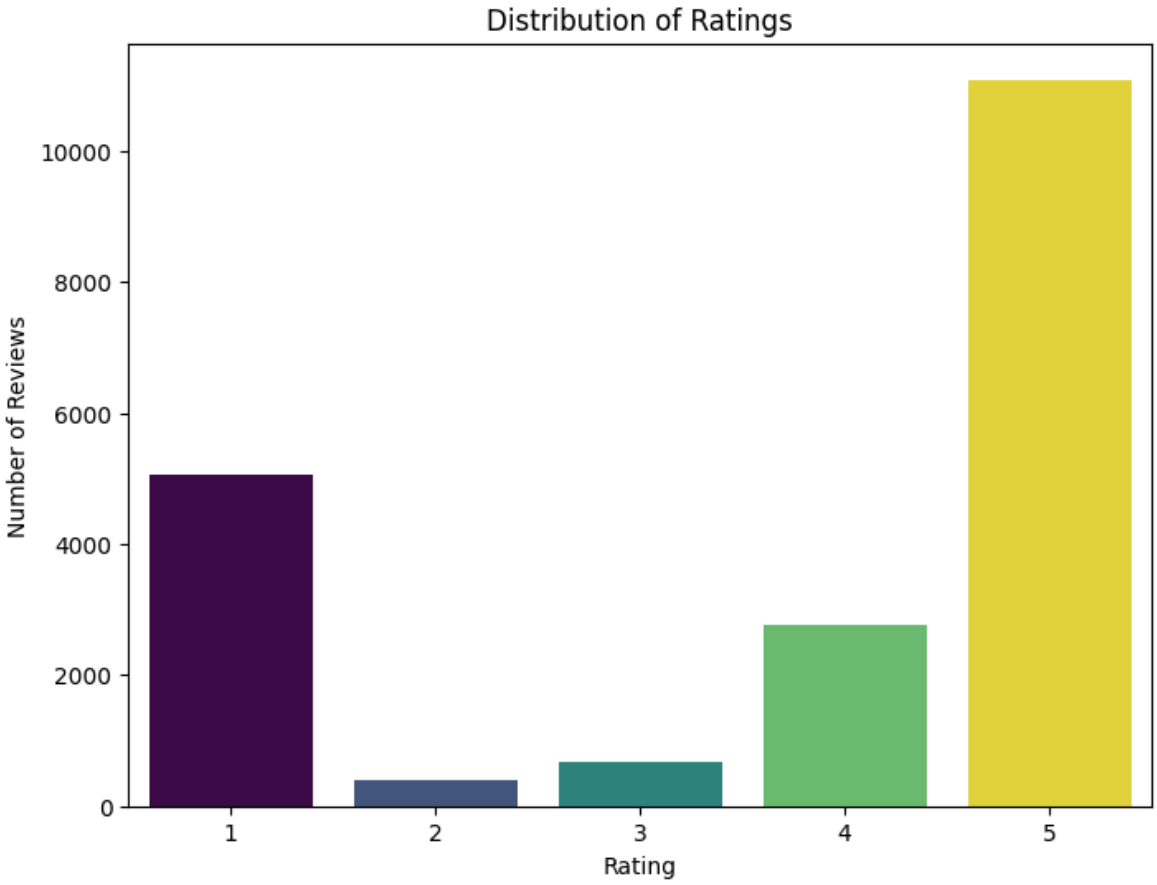
```
Number of duplicate rows before removal: 0
Number of duplicate rows after removal: 0
```

In [5]:
```
df['review_date'] = pd.to_datetime(df['review_date'])
print("Data type of 'review_date' after conversion:")
print(df['review_date'].dtype)
```

```
Data type of 'review_date' after conversion:
datetime64[ns]
```

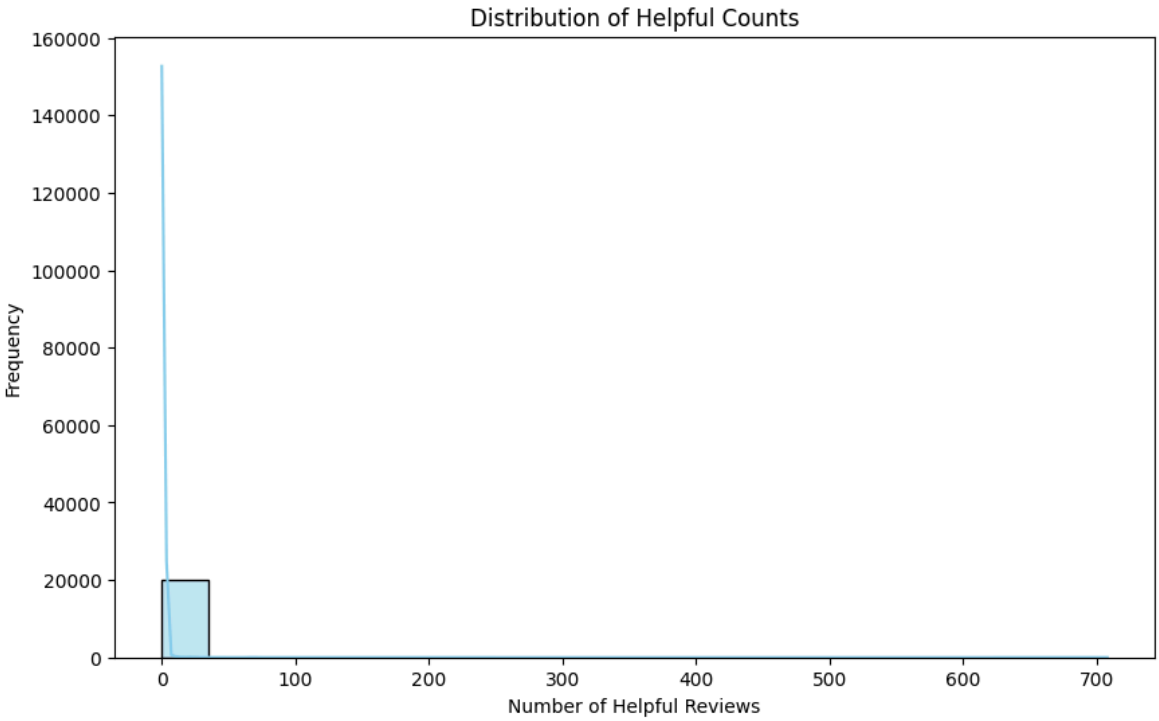
Univariate Analysis

In [38]:
```
plt.figure(figsize=(8, 6))
sns.countplot(
    data=df,
    x='rating',
    hue='rating',
    palette='viridis',
    legend=False
)
plt.title('Distribution of Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Reviews')
plt.show()

print("Count plot of 'rating' distribution displayed.")
```



Count plot of 'rating' distribution displayed.

In [39]:
```
plt.figure(figsize=(10, 6))
sns.histplot(df['helpful_count'], bins=20, kde=True, color='skyblue')
plt.title('Distribution of Helpful Counts')
plt.xlabel('Number of Helpful Reviews')
plt.ylabel('Frequency')
plt.show()

print("Histogram of 'helpful_count' distribution displayed.")
```



Histogram of 'helpful_count' distribution displayed.

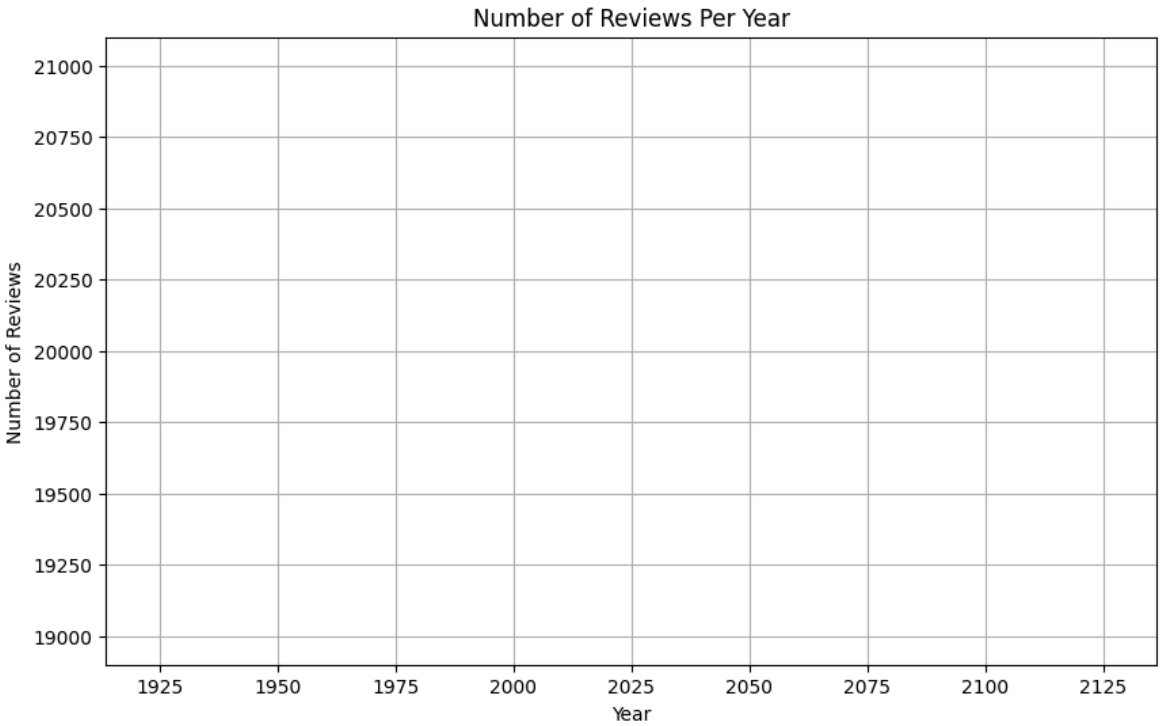
In [9]:
```
df['review_year'] = df['review_date'].dt.year
print("Created 'review_year' column.")
```

Created 'review_year' column.

In [10]:
```
reviews_per_year = df.groupby('review_year').size().reset_index(name='number_of_
print("Number of reviews per year calculated.")
```

Number of reviews per year calculated.

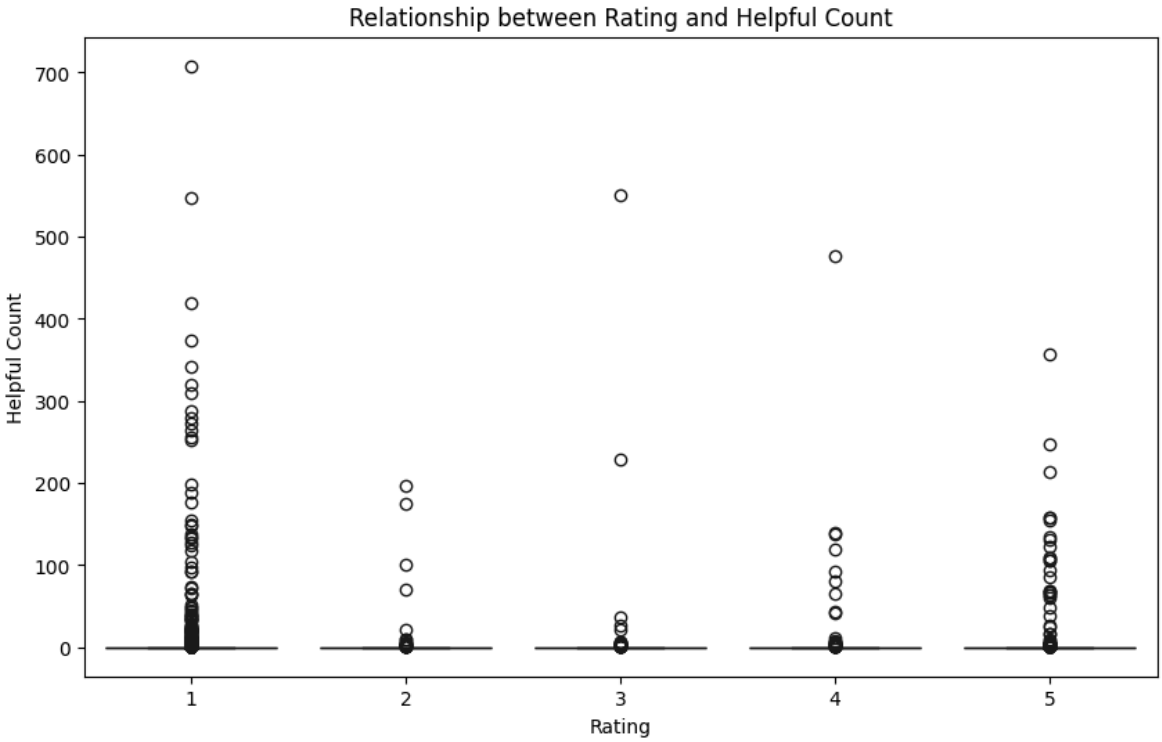
In [11]:
```
plt.figure(figsize=(10, 6))
sns.lineplot(x='review_year', y='number_of_reviews', data=reviews_per_year)
plt.title('Number of Reviews Per Year')
plt.xlabel('Year')
plt.ylabel('Number of Reviews')
plt.grid(True)
plt.show()
print("Line plot of reviews per year displayed.")
```



Line plot of reviews per year displayed.

Bivariate Analysis

In [13]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='rating', y='helpful_count', data=df, palette='viridis', hue='rati
plt.title('Relationship between Rating and Helpful Count')
plt.xlabel('Rating')
plt.ylabel('Helpful Count')
plt.show()
print("Box plot of 'rating' vs 'helpful_count' displayed.")
```

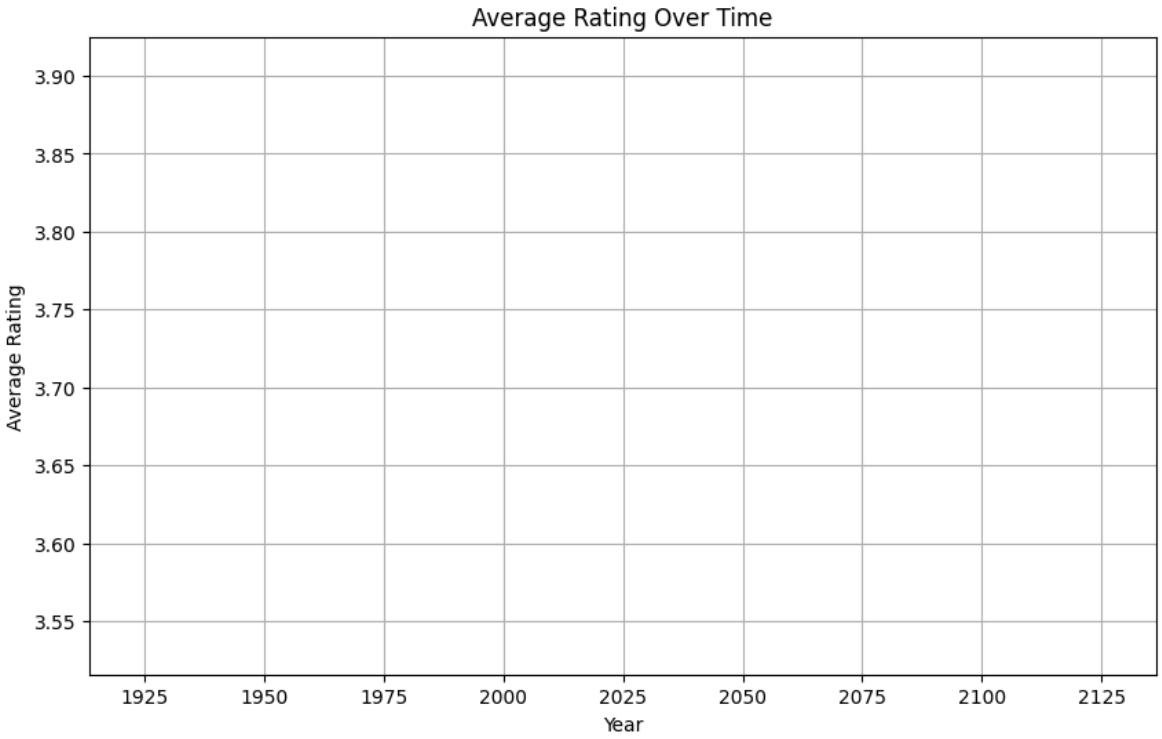


Box plot of 'rating' vs 'helpful_count' displayed.

In [14]:
```
avg_rating_per_year = df.groupby('review_year')['rating'].mean().reset_index()
print("Average rating per year calculated.")
```

Average rating per year calculated.

In [15]:
```
plt.figure(figsize=(10, 6))
sns.lineplot(x='review_year', y='rating', data=avg_rating_per_year)
plt.title('Average Rating Over Time')
plt.xlabel('Year')
plt.ylabel('Average Rating')
plt.grid(True)
plt.show()
print("Line plot of average rating over time displayed.")
```



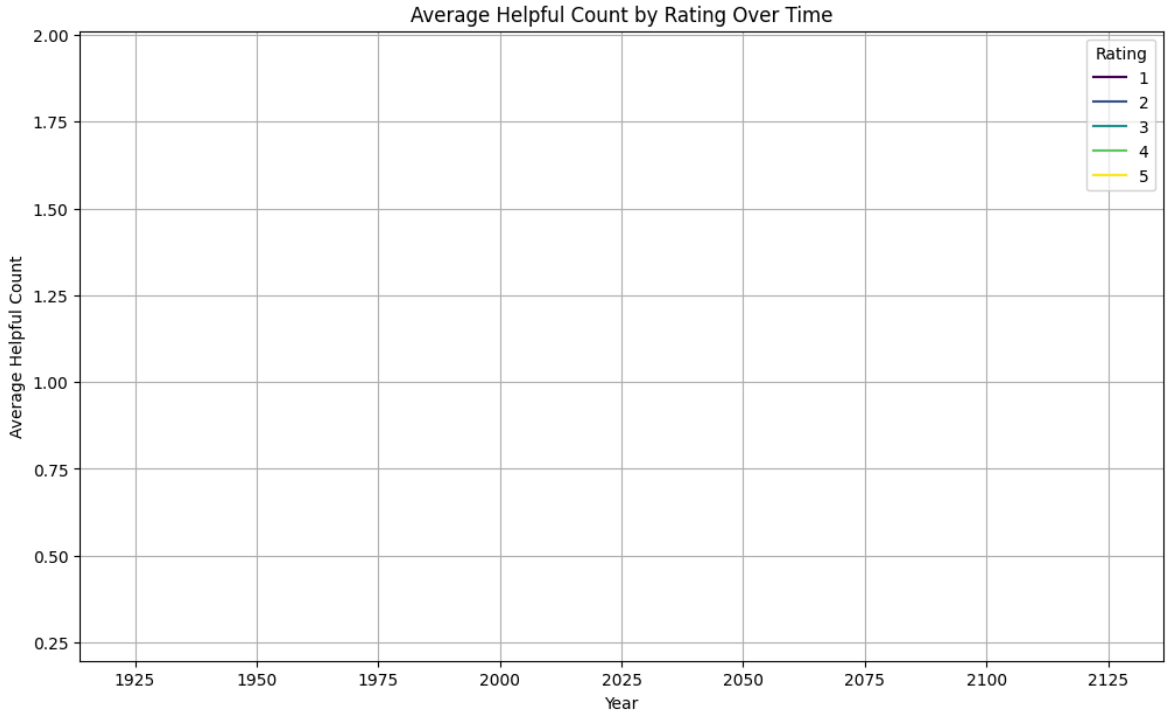
Line plot of average rating over time displayed.

Multivariate Analysis

In [16]:
```
avg_helpful_by_rating_year = df.groupby(['review_year', 'rating'])['helpful_coun
print("Average helpful count by rating and year calculated.")
```

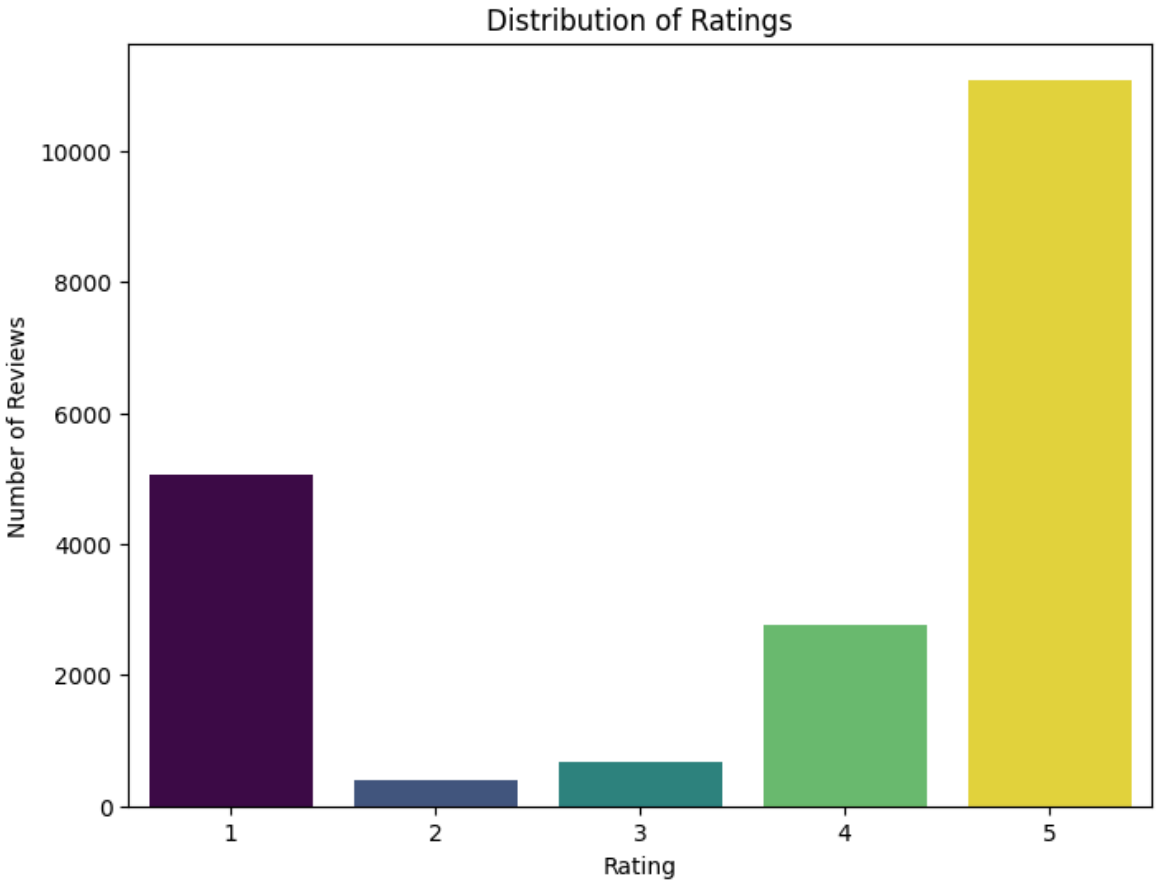
Average helpful count by rating and year calculated.

In [17]:
```
plt.figure(figsize=(12, 7))
sns.lineplot(x='review_year', y='helpful_count', hue='rating', data=avg_helpful_
plt.title('Average Helpful Count by Rating Over Time')
plt.xlabel('Year')
plt.ylabel('Average Helpful Count')
plt.grid(True)
plt.legend(title='Rating')
plt.show()
print("Line plot of average helpful count by rating over time displayed.")
```



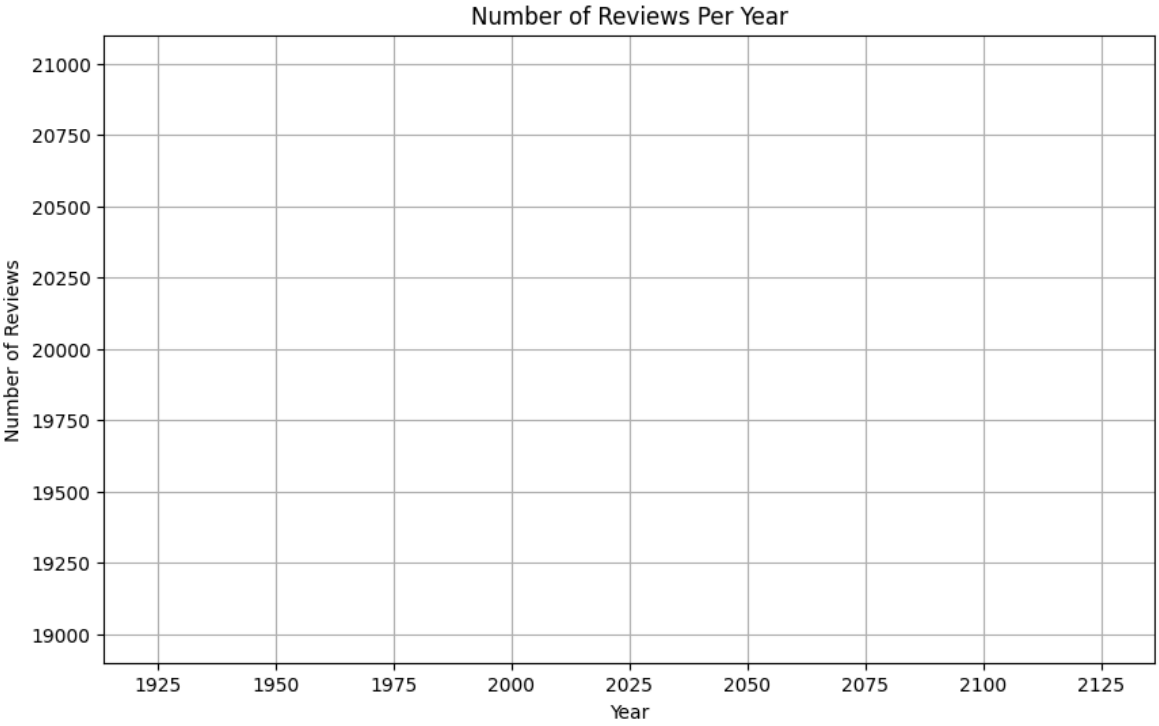
Line plot of average helpful count by rating over time displayed.

In [19]:
```
plt.figure(figsize=(8, 6))
sns.countplot(x='rating', data=df, palette='viridis', hue='rating', legend=False
plt.title('Distribution of Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Reviews')
plt.show()
print("Count plot of 'rating' distribution re-displayed.")
```



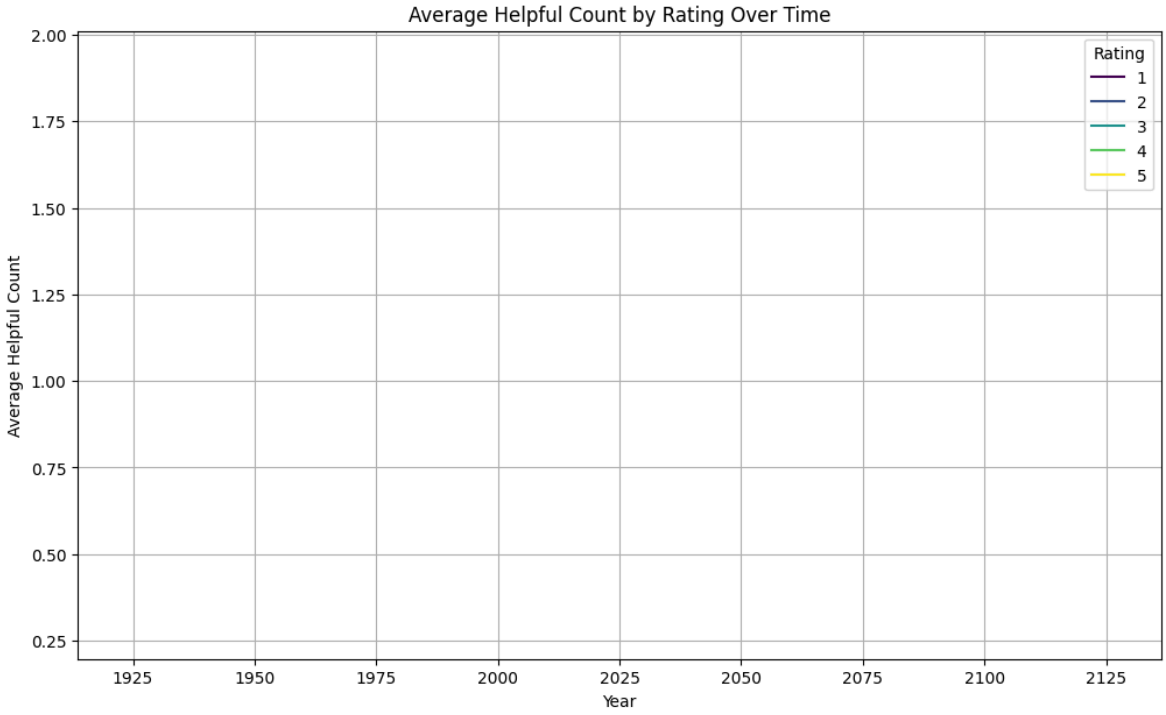
Count plot of 'rating' distribution re-displayed.

In [20]:
```
plt.figure(figsize=(10, 6))
sns.lineplot(x='review_year', y='number_of_reviews', data=reviews_per_year)
plt.title('Number of Reviews Per Year')
plt.xlabel('Year')
plt.ylabel('Number of Reviews')
plt.grid(True)
plt.show()
print("Line plot of reviews per year re-displayed.")
```



Line plot of reviews per year re-displayed.

In [21]:
```
plt.figure(figsize=(12, 7))
sns.lineplot(x='review_year', y='helpful_count', hue='rating', data=avg_helpful_
plt.title('Average Helpful Count by Rating Over Time')
plt.xlabel('Year')
plt.ylabel('Average Helpful Count')
plt.grid(True)
plt.legend(title='Rating')
plt.show()
print("Line plot of average helpful count by rating over time re-displayed.")
```



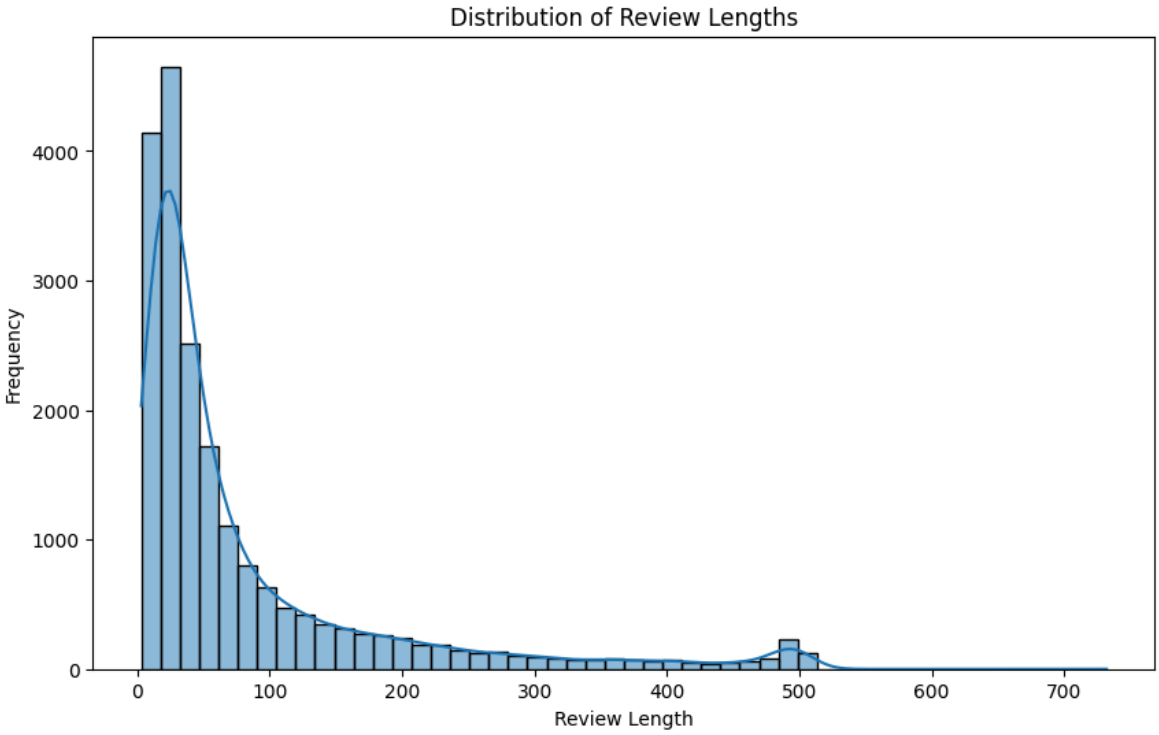
Line plot of average helpful count by rating over time re-displayed.

Feature Engineering

In [22]:
```
df['review_length'] = df['review_text'].str.len()
print("Created 'review_length' column.")
```

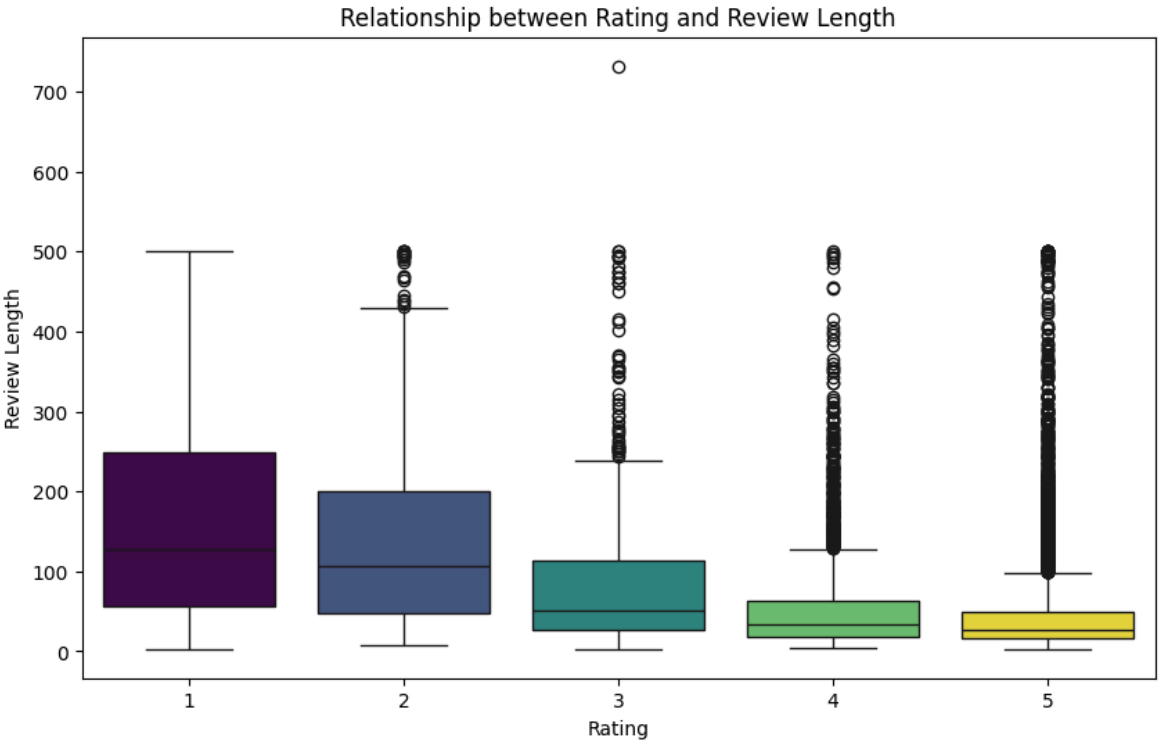
Created 'review_length' column.

In [23]:
```
plt.figure(figsize=(10, 6))
sns.histplot(df['review_length'], bins=50, kde=True)
plt.title('Distribution of Review Lengths')
plt.xlabel('Review Length')
plt.ylabel('Frequency')
plt.show()
print("Histogram of 'review_length' distribution displayed.")
```



Histogram of 'review_length' distribution displayed.

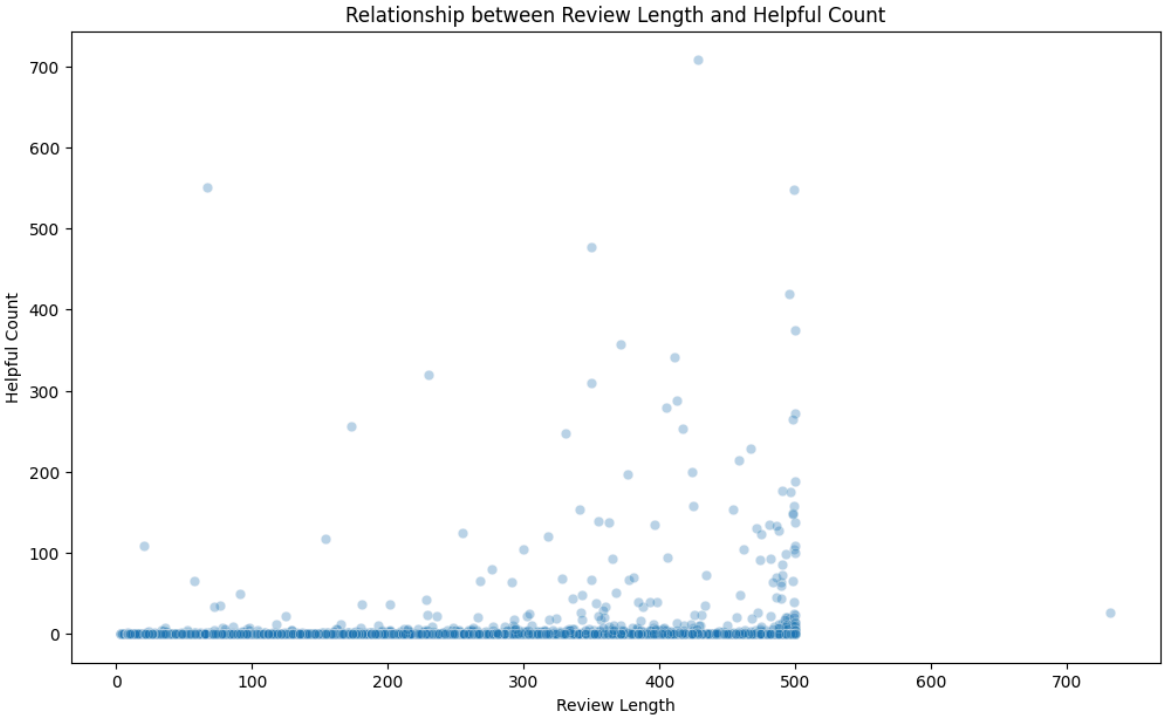
In [25]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='rating', y='review_length', data=df, palette='viridis', hue='rati
plt.title('Relationship between Rating and Review Length')
plt.xlabel('Rating')
plt.ylabel('Review Length')
plt.show()
print("Box plot of 'rating' vs 'review_length' displayed.")
```



Box plot of 'rating' vs 'review_length' displayed.

In [26]:
```
plt.figure(figsize=(12, 7))
sns.scatterplot(x='review_length', y='helpful_count', data=df, alpha=0.3)
plt.title('Relationship between Review Length and Helpful Count')
plt.xlabel('Review Length')
```

```
plt.ylabel('Helpful Count')
plt.show()
print("Scatter plot of 'review_length' vs 'helpful_count' displayed.")
```




```
Scatter plot of 'review_length' vs 'helpful_count' displayed.
```

Machine Learning

In [27]:
```
X = df[['helpful_count', 'review_year', 'review_length']]
y = df['rating']
print("Feature matrix X and target vector y defined.")
```

```
Feature matrix X and target vector y defined.
```

In [28]:
```
from sklearn.model_selection import train_test_split
print("Imported train_test_split from sklearn.model_selection.")
```

```
Imported train_test_split from sklearn.model_selection.
```

In [29]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
print("Shapes of the split datasets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
Shapes of the split datasets:
X_train shape: (16000, 3)
X_test shape: (4000, 3)
y_train shape: (16000,)
y_test shape: (4000,)
```

Implement and Evaluate Regression Models

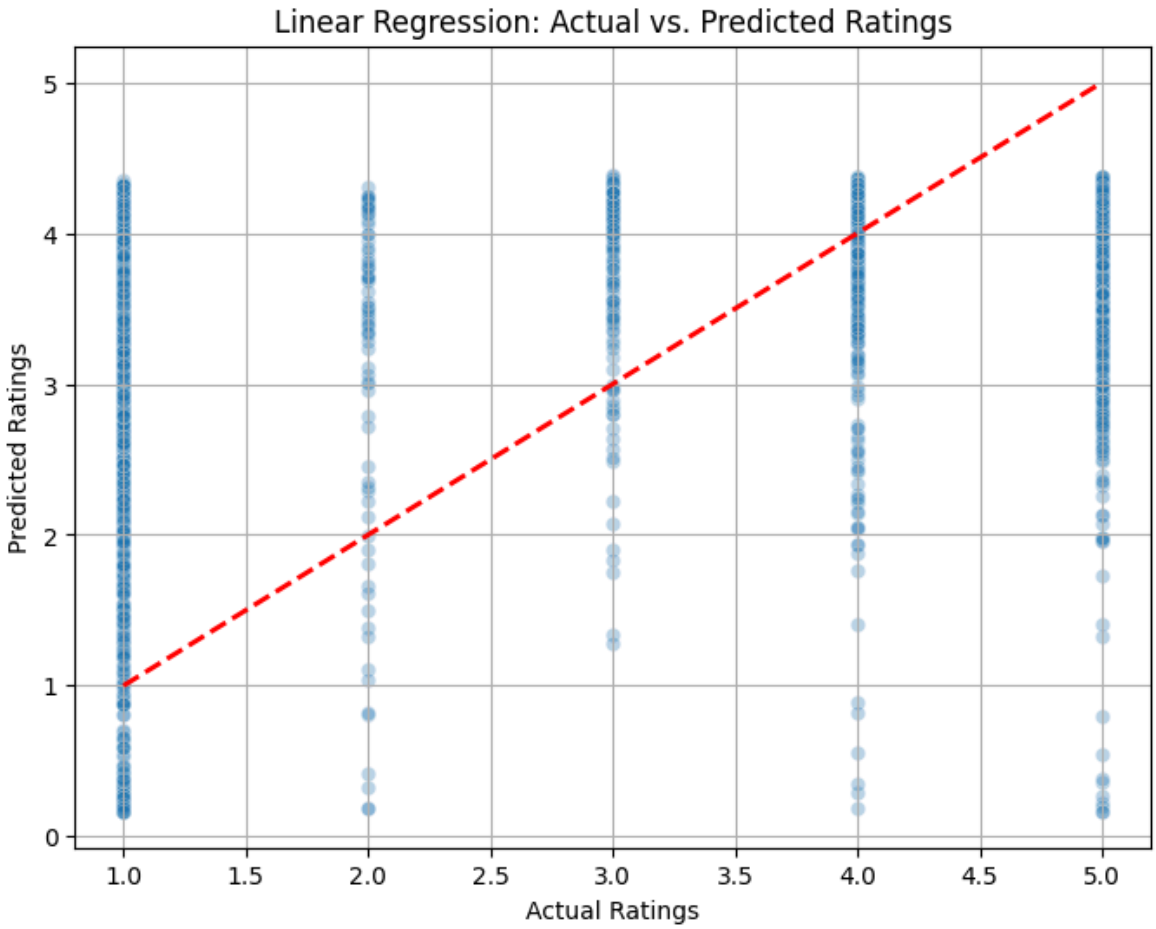
In [30]:
```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


linear_model = LinearRegression()
dtree_model = DecisionTreeRegressor(random_state=42)
rf_model = RandomForestRegressor(random_state=42)


print("Training Linear Regression model...")
linear_model.fit(X_train, y_train)
print("Training Decision Tree Regressor model...")
dtree_model.fit(X_train, y_train)
print("Training Random Forest Regressor model...")
rf_model.fit(X_train, y_train)


y_pred_linear = linear_model.predict(X_test)
y_pred_dtree = dtree_model.predict(X_test)
y_pred_rf = rf_model.predict(X_test)




mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)


mae_dtree = mean_absolute_error(y_test, y_pred_dtree)
mse_dtree = mean_squared_error(y_test, y_pred_dtree)
r2_dtree = r2_score(y_test, y_pred_dtree)


mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)


print("\n--- Model Performance --- ")
print("\nLinear Regression:")
print(f"  MAE: {mae_linear:.4f}")
print(f"  MSE: {mse_linear:.4f}")
print(f"  R-squared: {r2_linear:.4f}")

print("\nDecision Tree Regressor:")
print(f"  MAE: {mae_dtree:.4f}")
print(f"  MSE: {mse_dtree:.4f}")
print(f"  R-squared: {r2_dtree:.4f}")

print("\nRandom Forest Regressor:")
print(f"  MAE: {mae_rf:.4f}")
print(f"  MSE: {mse_rf:.4f}")
print(f"  R-squared: {r2_rf:.4f}")
```

```
Training Linear Regression model...
Training Decision Tree Regressor model...
Training Random Forest Regressor model...

--- Model Performance ---

Linear Regression:
  MAE: 1.1607
  MSE: 2.0675
  R-squared: 0.2748

Decision Tree Regressor:
  MAE: 1.0803
  MSE: 2.0556
  R-squared: 0.2790

Random Forest Regressor:
  MAE: 1.0781
  MSE: 1.9970
  R-squared: 0.2995
```

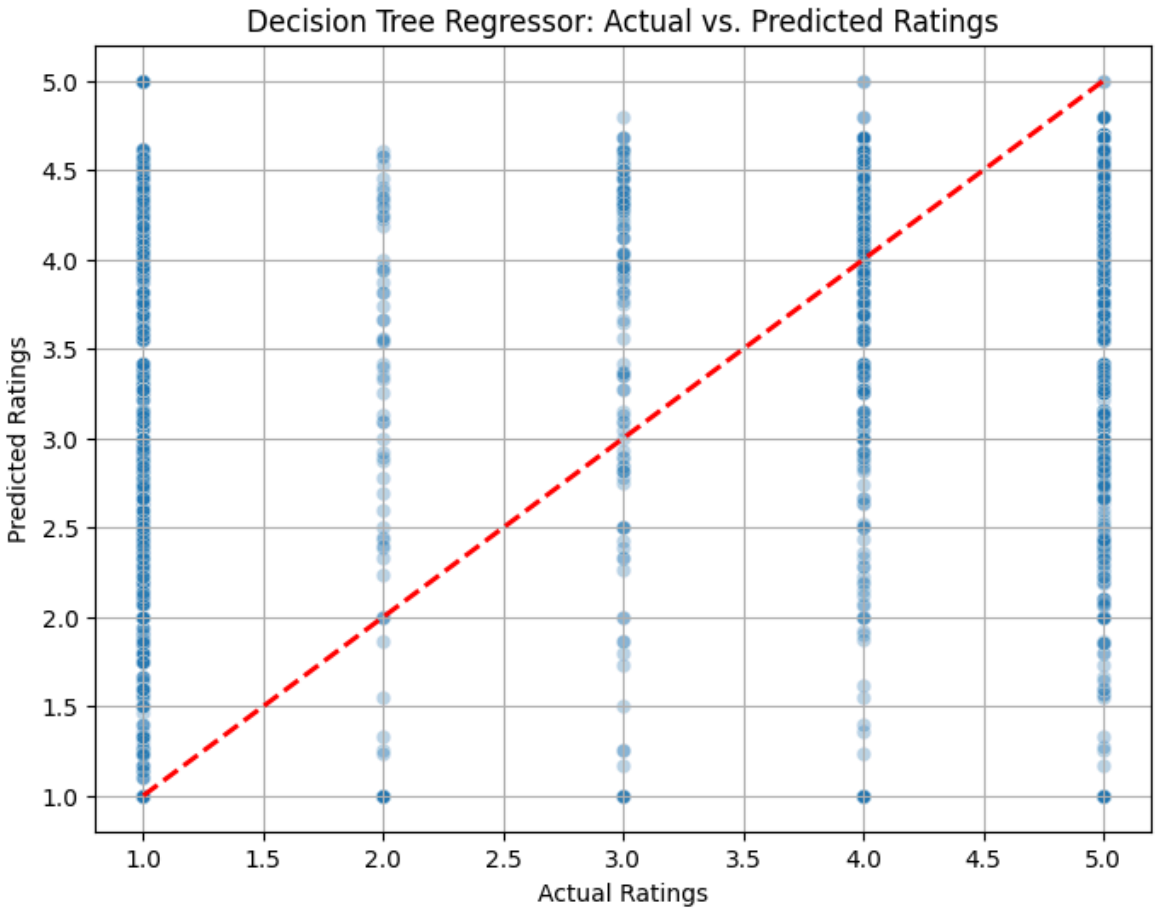
Visualize Regression Models Performance

In [31]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred_linear, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2
plt.title('Linear Regression: Actual vs. Predicted Ratings')
plt.xlabel('Actual Ratings')
plt.ylabel('Predicted Ratings')
plt.grid(True)
plt.show()
print("Scatter plot for Linear Regression (Actual vs. Predicted) displayed.")
```



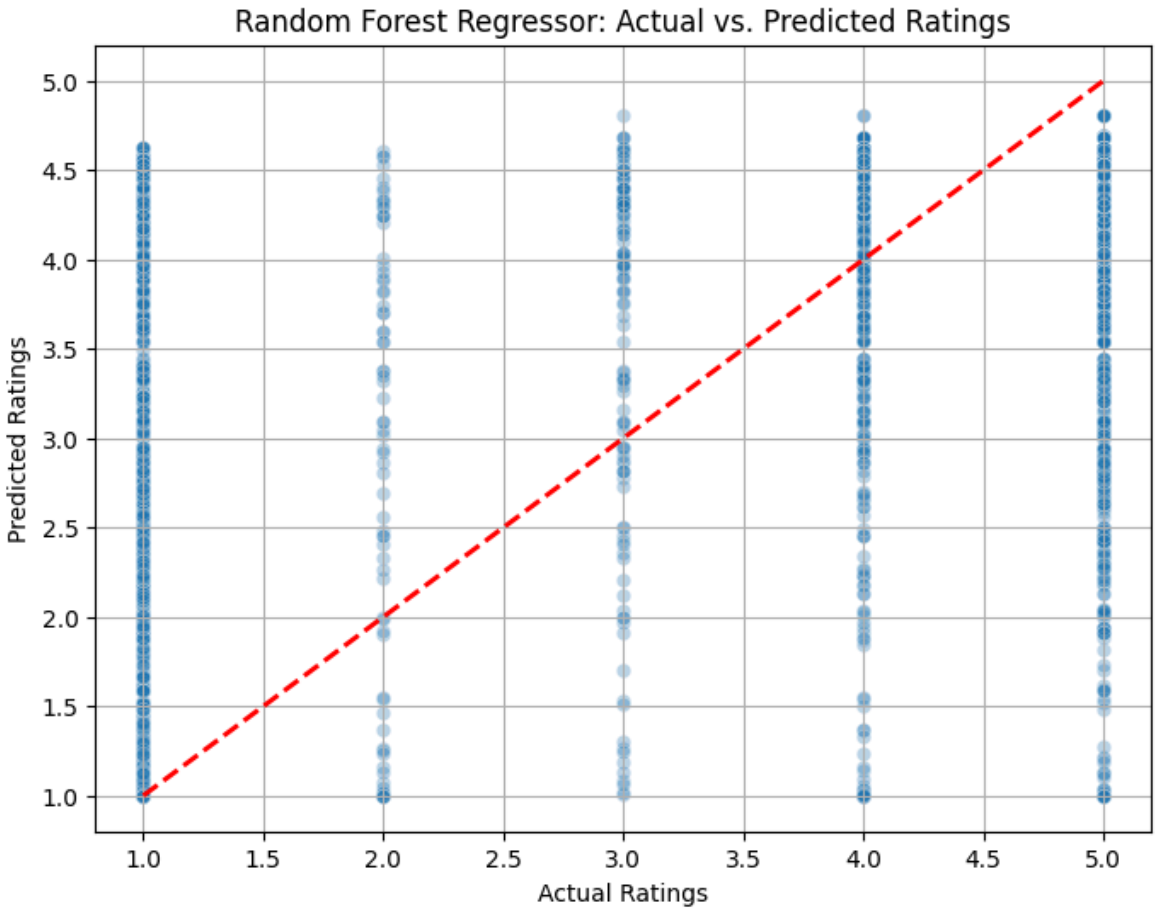
Scatter plot for Linear Regression (Actual vs. Predicted) displayed.

In [32]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred_dtree, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2
plt.title('Decision Tree Regressor: Actual vs. Predicted Ratings')
plt.xlabel('Actual Ratings')
plt.ylabel('Predicted Ratings')
plt.grid(True)
plt.show()
print("Scatter plot for Decision Tree Regressor (Actual vs. Predicted) displayed
```



Scatter plot for Decision Tree Regressor (Actual vs. Predicted) displayed.

In [33]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred_rf, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2
plt.title('Random Forest Regressor: Actual vs. Predicted Ratings')
plt.xlabel('Actual Ratings')
plt.ylabel('Predicted Ratings')
plt.grid(True)
plt.show()
print("Scatter plot for Random Forest Regressor (Actual vs. Predicted) displayed
```



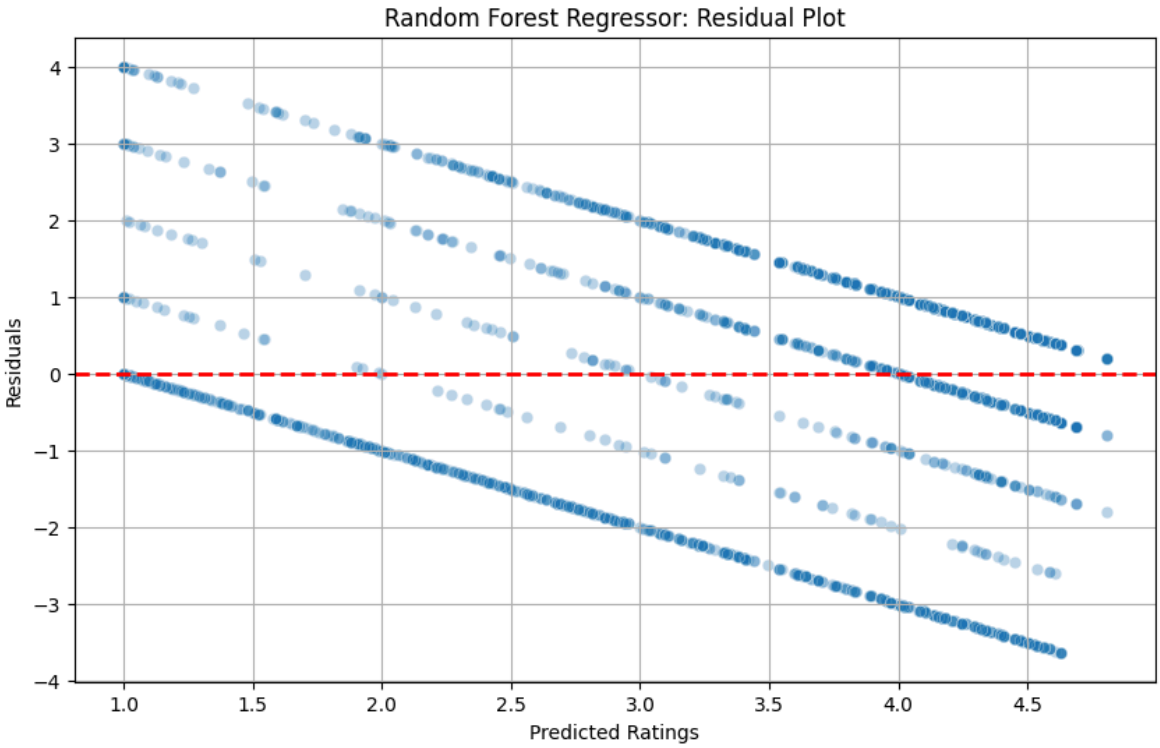

```
Scatter plot for Random Forest Regressor (Actual vs. Predicted) displayed.
```

Select and Visualize Best Regression Model

In [34]:
```
residuals_rf = y_test - y_pred_rf
print("Residuals for Random Forest Regressor calculated.")
```

```
Residuals for Random Forest Regressor calculated.
```

In [35]:
```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_rf, y=residuals_rf, alpha=0.3)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.title('Random Forest Regressor: Residual Plot')
plt.xlabel('Predicted Ratings')
plt.ylabel('Residuals')
plt.grid(True)
plt.show()
print("Residual plot for Random Forest Regressor displayed.")
```

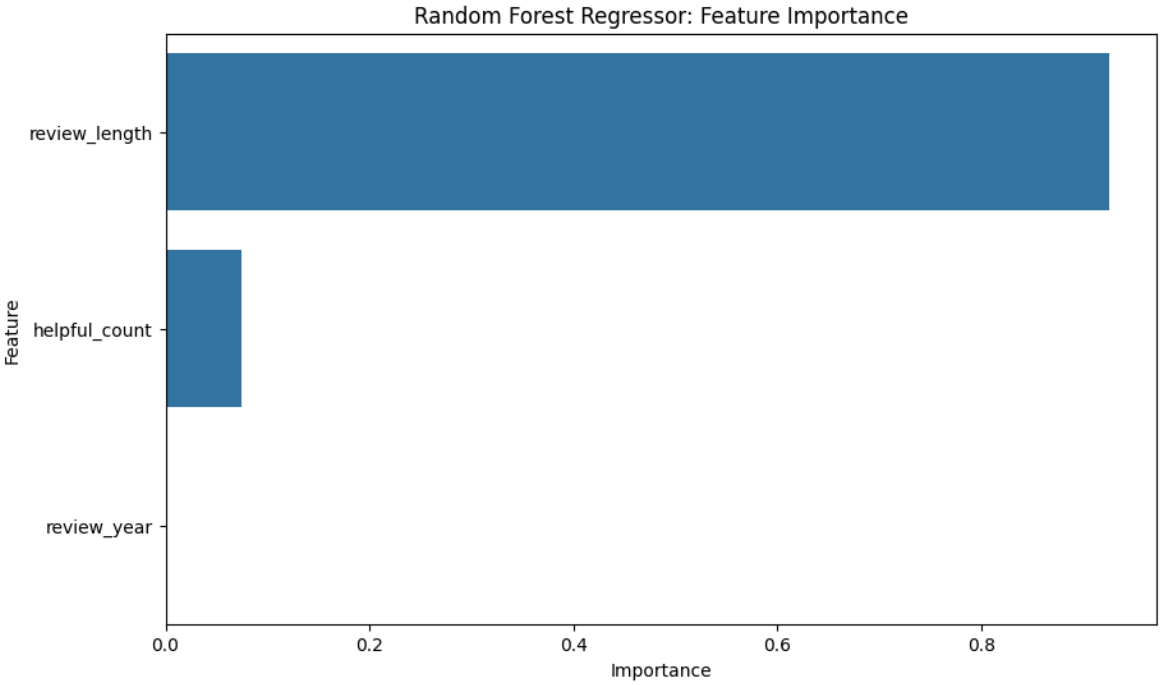


Residual plot for Random Forest Regressor displayed.

In [36]:
```
feature_importances = rf_model.feature_importances_
feature_names = X.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': feature_im
importance_df = importance_df.sort_values(by='Importance', ascending=False)
print("Feature importances extracted for Random Forest Regressor.")
```

Feature importances extracted for Random Forest Regressor.

In [37]:
```
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title('Random Forest Regressor: Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()
print("Bar plot of feature importances for Random Forest Regressor displayed.")
```



Bar plot of feature importances for Random Forest Regressor displayed.